Local-Consistency Widgetization: Direct Manipulation of Structured Parameters in LLM Responses

Aftab Gazali
Université de Montréal
Montréal, Canada

Xinyan Guo
Université de Montréal
Montréal, Canada

Zhichao Ye
Université de Montréal
Montréal, Canada

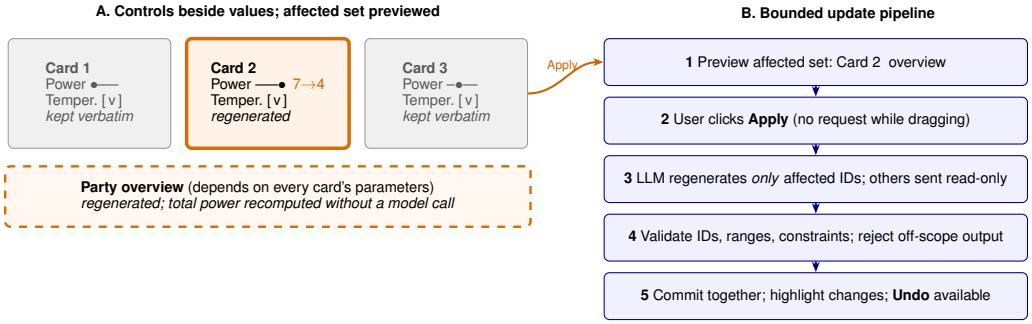


Figure 1: System sketch (party-builder demo). Editing Card 2's power level marks Card 2 and the party overview as the dependency closure; Cards 1 and 3 never pass through the model and remain unchanged in application state.

Abstract

Refining an LLM-generated artifact in a chat interface is costly: each small change must be re-expressed as a prompt, and users must then inspect the regenerated output for unintended changes [7]. Widget-based systems expose parameters as controls [5, 8], but typically leave open which parts of the stored output an edit may replace. We propose *Local-Consistency Widgetization*, a schema-driven approach that binds controls to persistent data nodes and a developer-authored dependency graph, so that an edit replaces only its dependent nodes and every other node is preserved verbatim. We will demonstrate it in a tabletop-RPG party builder: users adjust a character's power level (slider) or temperament (dropdown), preview the affected cards and party overview, apply the change, see what changed highlighted, and undo it, following direct-manipulation principles of visibility, incremental feedback, and reversibility [4, 6]. A formative within-subjects study will compare bounded updates with whole-response regeneration under identical widgets, examining perceived predictability, control, and re-inspection of unaffected content. We expect to contribute a reusable update mechanism and design evidence on when preserving update scope helps or hinders LLM-assisted editing.

1 Introduction

Imagine a game master who asks an LLM for a three-character adventuring party, accepts it, and then wants the second character to be weaker. In a chat interface the user must phrase this as a prompt, and the model may return a party in which the other characters' abilities and backstories have also changed. Even when those changes are plausible, the user must compare versions to find what moved, a *gulf of evaluation* [4] that grows with the size of the output. The same problem arises whenever people iteratively refine structured, multi-part LLM output such as itineraries, lesson plans, or product specifications. A small, targeted edit should have a visible, understandable scope, and content the user has already accepted should stay stable outside it.

Why existing solutions are not enough. Non-expert users struggle to express and test desired behavior through prompts [7]. DirectGPT lets users manipulate generated objects directly, but translates manipulation back into prompts [5]. Malleable Prompting turns preference expressions in the prompt into widgets and shows their influence on the output [8]. JELLY generates task-driven data models, including schemas and dependencies, to support malleable interfaces [2]. As far as we can tell, none of these makes a *guaranteed scope of replacement* its central design goal: a control may change a parameter, yet nothing prevents the regenerated response from altering content the user did not intend to touch.

Our solution. Fig. 1 sketches our approach. An LLM fills a developer-defined JSON schema of nodes (character cards and a party overview), each with a stable identifier, parameters, and prose. Developers declare once which parameters affect which nodes; the affected set is every node reachable through these dependencies. When a user adjusts a control, the interface previews the affected set. After the user clicks Apply, the application asks the model to regenerate only those nodes, supplying unaffected nodes as read-only context, rejects replacements for any other identifier, validates ranges, and commits valid replacements together, with changes highlighted and undo available. Unaffected nodes never pass through the model, so their preservation does not depend on model compliance. We call this property *local consistency*: stored content outside the dependency closure is unchanged. It does not guarantee that the result is semantically coherent or balanced as a game.

The design follows Shneiderman's direct-manipulation principles [6]: continuously visible objects, controls beside the values they change, rapid incremental actions, and reversibility. Scope previews and highlighted changes aim to narrow the gulf of evaluation [4] by telling users where change will occur and afterwards what changed. We chose a tabletop-RPG party builder as the demonstration because it is playful, has naturally independent units (characters) plus a cross-node aggregate (the party overview), and lets participants judge

whether untouched characters changed without domain expertise. The mechanism is schema-driven, so itineraries or other multi-part outputs are further instances of the same pattern; we will evaluate one domain only.

Planned evaluation. A formative within-subjects study with 8–10 LLM-chat users will compare two conditions that share widgets, layout, progress feedback, undo, initial party, and model, and differ only in update scope: bounded update versus regeneration of the complete structured response. We ask: **RQ1**, does bounded updating improve perceived predictability and control and reduce re-inspection of unaffected cards? **RQ2**, what costs does it introduce, such as missed dependencies or confusion about scope? We will measure off-scope changes, completion time, latency, ratings, hover/focus dwell on unaffected cards, and retrospective interview themes, and report descriptive differences rather than claim statistical significance.

2 Related Work

Prompting and direct manipulation. Zamfirescu-Pereira et al. [7] show that non-experts iterate on prompts unsystematically, which motivates reducing the need to translate every change into language. Shneiderman [6] and Hutchins et al. [4] give the principles and the gulfs of execution and evaluation we use to frame the design. DirectGPT [5] applies these ideas to LLMs by turning manipulation of generated objects into engineered prompts. We build on this interaction style but enforce update scope through application state instead of relying on the prompt alone.

Widgets and exploration for generation. Malleable Prompting [8] converts prompt-side preference expressions into widgets and adjusts decoding accordingly; we instead bind *output-side* controls to persistent nodes and use ordinary structured generation. Spellburst [1] offers a node-based canvas with branching and merging to explore variations of generated creative code, and TaleBrush [3] lets users sketch a protagonist's fortune to steer generated stories. These systems support *exploring* alternatives; we target the complementary task of *refining* an already accepted output, where unintended change is costly. Our game-themed demo shares TaleBrush's narrative flavor but replaces sketching with discrete, scoped controls.

Data models and generative interfaces. JELLY [2] generates evolving task-driven data models, including schemas and dependencies, for malleable interfaces. We adopt its separation of data from presentation but use a developer-authored schema for one domain, with a deliberately constrained update policy: dependencies determine which nodes may be replaced, and all others are retained exactly. VISAR [9] keeps a visual-programming graph synchronized with text drafts to support argumentative writing; we use dependency structure not to plan or draft, but to bound which stored text an edit may replace. We make no claim to introduce widgets or dependency graphs; our contribution is the bounded update policy and a study of its benefits and costs.

3 Scope and Expected Contribution

Manually authored dependencies require developer effort and may omit indirect relationships; a missed dependency can leave preserved cards inconsistent with regenerated ones, so verbatim preservation is narrower than semantic consistency, and broad dependencies may force most of the response to change. Schema and range checks cannot establish game balance or narrative quality, and local updating may preserve flaws in accepted content or block a better global revision. The prototype will not support free-form follow-up prompts, automatic dependency discovery, or dynamically generated widgets. An 8–10 person convenience sample, brief tasks, one model, and one domain give formative evidence with limited generalizability, and hover/focus dwell is not a measure of gaze or proof of re-reading. Exact preservation is an implementation property rather than an HCI contribution in itself, so the study will focus on whether users understand update scope and can verify changes with less effort, including cases where locality confuses them or offers no benefit. We expect to deliver (i) a bounded-update mechanism and prototype, and (ii) formative evidence about the benefits and costs of preserving update scope in LLM-assisted editing.

References

[1] Tyler Angert, Miroslav Suzara, Jenny Han, Christopher Pondoc, and Hariharan Subramonyam. 2023. Spellburst: A Node-based Interface for Exploratory Creative Coding with Natural Language Prompts. In *Proc. UIST '23*. ACM. doi:10.1145/3586183.3606719

[2] Yining Cao, Peiling Jiang, and Haijun Xia. 2025. Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model. In *Proc. CHI '25*. ACM. doi:10.1145/3706598.3713285

[3] John Joon Young Chung, Wooseok Kim, Kang Min Yoo, Hwaran Lee, Eytan Adar, and Minsuk Chang. 2022. TaleBrush: Sketching Stories with Generative Pretrained Language Models. In *Proc. CHI '22*. ACM. doi:10.1145/3491102.3501819

[4] Edwin L. Hutchins, James D. Hollan, and Donald A. Norman. 1985. Direct Manipulation Interfaces. *Human–Computer Interaction* 1, 4 (1985), 311–338.

[5] Damien Masson, Sylvain Malacria, Géry Casiez, and Daniel Vogel. 2024. DirectGPT: A Direct Manipulation Interface to Interact with Large Language Models. In *Proc. CHI '24*. ACM. doi:10.1145/3613904.3642462

[6] Ben Shneiderman. 1983. Direct Manipulation: A Step Beyond Programming Languages. *Computer* 16, 8 (1983), 57–69. doi:10.1109/MC.1983.1654471

[7] J.D. Zamfirescu-Pereira, Richmond Y. Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts. In *Proc. CHI '23*. ACM. doi:10.1145/3544548.3581388

[8] Chao Zhang, Yiren Liu, Lunyiu Nie, Jeffrey M. Rzeszotarski, Yun Huang, and Tal August. 2026. From Words to Widgets for Controllable LLM Generation. arXiv:2604.10925.

[9] Zheng Zhang, Jie Gao, Ranjodh Singh Dhaliwal, and Toby Jia-Jun Li. 2023. VISAR: A Human-AI Argumentative Writing Assistant with Visual Programming and Rapid Draft Prototyping. In *Proc. UIST '23*. ACM. doi:10.1145/3586183.3606800